QUERY PROCESSING LAB EXP 1 to 10

NAME : UDHAYAKUMAR U
REG NO : 192424332
COURSE CODE : DSA0503 - Query Processing

EXP 1

Aim

To write a Python program using Pandas to select and display the distinct Department IDs from the given employee/department data.

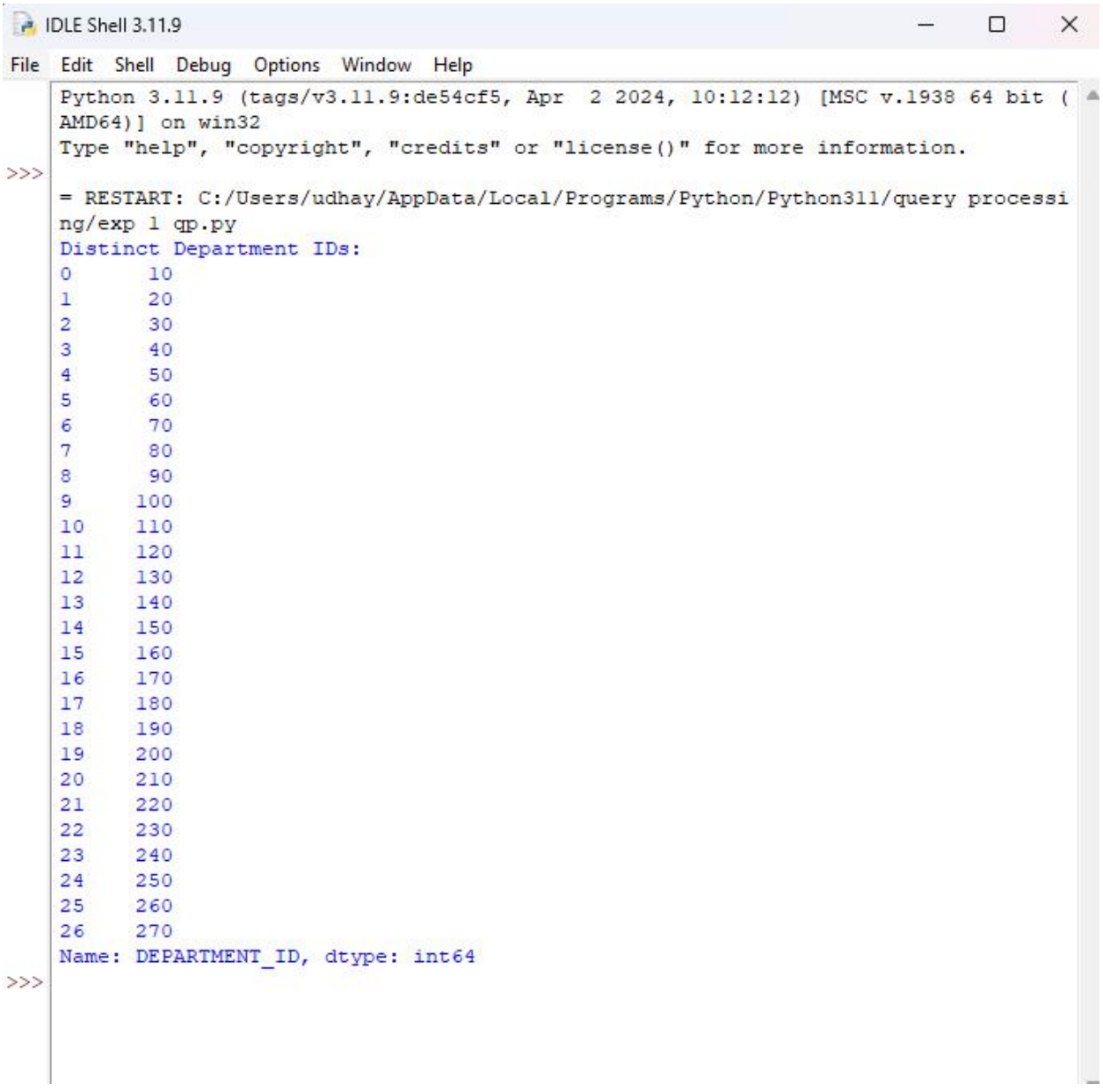
Algorithm

- Start the program.
- Import the **Pandas** library.
- Create a DataFrame with the given Department IDs.
- Select the **DEPARTMENT_ID** column.
- Use the `drop_duplicates()` or `unique()` method to get distinct Department IDs.
- Display the distinct Department IDs.
- Stop the program.

Code

```
import pandas as pd

# Create the DataFrame
data = {
   "DEPARTMENT_ID": [10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110,
                     120, 130, 140, 150, 160, 170, 180, 190, 200,
                     210, 220, 230, 240, 250, 260, 270],
   "DEPARTMENT_NAME": [
      "Administration", "Marketing", "Purchasing", "Human Resources",
      "Shipping", "IT", "Public Relations", "Sales", "Executive",
      "Finance", "Accounting", "Treasury", "Corporate Tax",
      "Control And Credit", "Shareholder Services", "Benefits",
      "Manufacturing", "Construction", "Contracting", "Operations",
      "IT Support", "NOC", "IT Helpdesk", "Government Sales",
      "Retail Sales", "Recruiting", "Payroll"
   ],
   "MANAGER_ID": [200, 201, 114, 203, 121, 103, 204, 145, 100, 108,
                  205, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
   "LOCATION_ID": [1700, 1800, 1700, 2400, 1500, 1400, 2700, 2500,
                   1700, 1700, 1700, 1700, 1700, 1700, 1700, 1700,
                   1700, 1700, 1700, 1700, 1700, 1700, 1700, 1700,
                   1700, 1700, 1700]
}

df = pd.DataFrame(data)

# Select distinct Department IDs
distinct_ids = df["DEPARTMENT_ID"].drop_duplicates()

# Display the result
print("Distinct Department IDs:")
print(distinct_ids)
```

OUTPUT


IDLE Shell 3.11.9
File Edit Shell Debug Options Window Help
Python 3.11.9 (tags/v3.11.9:de54cf5, Apr 2 2024, 10:12:12) [MSC v.1938 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/query processing/exp 1 qp.py
Distinct Department IDs:
0 10
1 20
2 30
3 40
4 50
5 60
6 70
7 80
8 90
9 100
10 110
11 120
12 130
13 140
14 150
15 160
16 170
17 180
18 190
19 200
20 210
21 220
22 230
23 240
24 250
25 260
26 270
Name: DEPARTMENT_ID, dtype: int64
>>>

EXP 2

Aim

To write a Python program using Pandas to display the Employee IDs of employees who have done two or more jobs in the past.

Algorithm

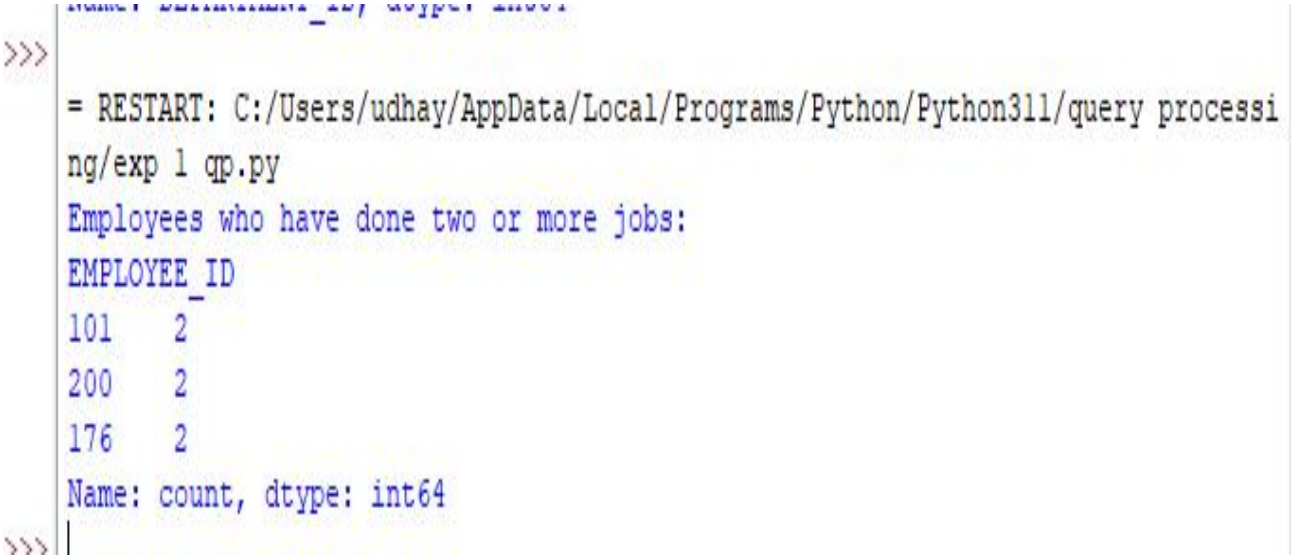
- ❖ Start the program.
- ❖ Import the Pandas library.
- ❖ Create a DataFrame with the employee job history.
- ❖ Group the data by EMPLOYEE_ID.
- ❖ Count the number of jobs for each employee.
- ❖ Select employees whose job count is 2 or more.
- ❖ Display the Employee IDs.
- ❖ Stop the program.

CODE

```
import pandas as pd

# Employee data

df = pd.DataFrame({

   "EMPLOYEE_ID": [102, 101, 101, 201, 114, 122, 200, 176, 176, 200]

}

# Display employees with two or more jobs

result = df["EMPLOYEE_ID"].value_counts()
```

```
print("Employees who have done two or more jobs:")
print(result[result >= 2])
```

OUTPUT



EXP 3

Aim

To write a Python program using Pandas to display the job details in descending order of Job Title.

Algorithm

- ❖ Start the program.
- ❖ Import the Pandas library.
- ❖ Create a DataFrame with the job details.
- ❖ Sort the DataFrame by the JOB_TITLE column in descending order.
- ❖ Display the sorted job details.
- ❖ Stop the program.

CODE

```
import pandas as pd
# Job data
df = pd.DataFrame({
   "JOB_ID":      ["AD_PRES",      "AD_VP",      "AD_ASST",      "FI_MGR",
"FI_ACCOUNT"],
   "JOB_TITLE": ["President",
                 "Administration Vice President",
                 "Administration Assistant",
                 "Finance Manager",
                 "Accountant"]
})
# Sort by JOB_TITLE in descending order
result = df.sort_values(by="JOB_TITLE", ascending=False)
print(result)
```

OUTPUT

```
>>>
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/query processi
ng/exp 1 qp.py
       JOB_ID                     JOB_TITLE
0     AD_PRES                     President
3      FI_MGR               Finance Manager
1       AD_VP  Administration Vice President
2     AD_ASST       Administration Assistant
4  FI_ACCOUNT                    Accountant
>>>
```

EXP 4

Aim

To write a Python program using **Pandas** to create a **line plot** of the historical stock prices of **Alphabet Inc.** between two specific dates.

Algorithm

1. Start the program.
2. Import the **Pandas** and **Matplotlib** libraries.
3. Create a DataFrame with sample stock price data.
4. Convert the **Date** column to datetime format.
5. Plot the **Close Price** against the **Date** using a line graph.
6. Add the title and axis labels.
7. Display the graph.
8. Stop the program.

CODE

```
import pandas as pd

import matplotlib.pyplot as plt



# Sample stock price data

data = {

    "Date": ["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04", "2024-01-05"],
```

```
    "Close": [140, 142, 141, 145, 147]
}

df = pd.DataFrame(data)

# Convert Date column to datetime
df["Date"] = pd.to_datetime(df["Date"])

# Create line plot
plt.plot(df["Date"], df["Close"], marker='o')

plt.title("Alphabet Inc. Stock Prices")
plt.xlabel("Date")
plt.ylabel("Closing Price")
plt.grid(True)

plt.show()
```

OUTPUT


Alphabet Inc. Stock Prices
Closing Price
147
146
145
144
143
142
141
140
01-01 00
01-01 12
01-02 00
01-02 12
01-03 00
01-03 12
01-04 00
01-04 12
01-05 00
Date

EXP 5

Aim

To write a Python program using Pandas to create a bar plot of the trading volume of Alphabet Inc. stock between two specific dates.

Algorithm

- ❖ Start the program.
- ❖ Import the Pandas and Matplotlib libraries.
- ❖ Create a DataFrame with the Date and Trading Volume.
- ❖ Convert the Date column to datetime format.
- ❖ Create a bar plot using the Date and Volume columns.
- ❖ Add the title and axis labels.
- ❖ Display the graph.
- ❖ Stop the program.

CODE

```
import pandas as pd

import matplotlib.pyplot as plt



# Sample stock trading volume data

data = {

   "Date": ["2024-01-01", "2024-01-02", "2024-01-03", "2024-01-04", "2024-01-05"],

   "Volume": [1200000, 1500000, 1000000, 1800000, 1600000]

}
```

```
df = pd.DataFrame(data)


# Convert Date column to datetime

df["Date"] = pd.to_datetime(df["Date"])


# Create bar plot

plt.bar(df["Date"].dt.strftime("%Y-%m-%d"), df["Volume"])


plt.title("Alphabet Inc. Trading Volume")

plt.xlabel("Date")

plt.ylabel("Trading Volume")


plt.show()
```

OUTPUT

EXP 6

Aim

To write a Python program using Pandas to create a scatter plot of the trading Volume and Closing Stock Price of Alphabet Inc. stock between two specific dates.

Algorithm

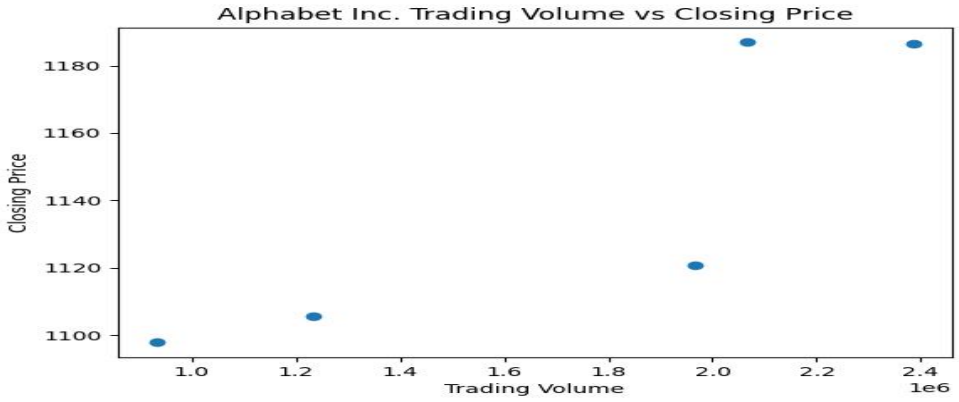
- ❖ Start the program.
- ❖ Import the Pandas and Matplotlib libraries.
- ❖ Create a DataFrame with Date, Close, and Volume.
- ❖ Create a scatter plot using Volume on the X-axis and Close Price on the Y-axis.
- ❖ Add the title and axis labels.
- ❖ Display the scatter plot.
- ❖ Stop the program.

CODE

```
import pandas as pd

import matplotlib.pyplot as plt



# Sample stock data

data = {

    "Date": ["01-04-2020", "02-04-2020", "03-04-2020", "06-04-2020", "07-04-2020"],

    "Close": [1105.62, 1120.84, 1097.86, 1186.92, 1186.51],
```

```
    "Volume": [1234100, 1964900, 931500, 2064700, 2387300]
}

df = pd.DataFrame(data)

# Scatter Plot
plt.scatter(df["Volume"], df["Close"])

plt.title("Alphabet Inc. Trading Volume vs Closing Price")
plt.xlabel("Trading Volume")
plt.ylabel("Closing Price")

plt.show()
```

OUTPUT



EXP 7

Aim

To write a Python program using Pandas to create a Pivot Table and find the maximum and minimum sale values of the items.

Algorithm

- ❖ Start the program.
- ❖ Import the Pandas library.
- ❖ Create a DataFrame with Item and Sale columns.
- ❖ Create a Pivot Table using *pivot_table()*.
- ❖ Apply max and min aggregation functions to the Sale column.
- ❖ Display the Pivot Table.
- ❖ Stop the program.

CODE

```
import pandas as pd

# Sample sales data

data = {

   "Item": ["Pen", "Pen", "Book", "Book", "Pencil", "Pencil"],

   "Sale": [100, 150, 300, 250, 80, 120]

}


df = pd.DataFrame(data)
```

```
# Create Pivot Table
pivot = pd.pivot_table(
    df,
    values="Sale",
    index="Item",
    aggfunc=["max", "min"]
)
print(pivot)
```

OUTPUT

```
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/query processi
ng/exp 1 qp.py
        max  min
       Sale Sale
Item
Book    300  250
Pen     150  100
Pencil  120   80
```

EXP 8

Aim

To write a Python program using Pandas to create a Pivot Table and find the item-wise units sold.

Algorithm

- ❖ Start the program.
- ❖ Import the Pandas library.
- ❖ Create a DataFrame with Item and Units Sold.
- ❖ Create a Pivot Table using the sum function.
- ❖ Display the item-wise units sold.
- ❖ Stop the program.

CODE

```
import pandas as pd

# Sample sales data
data = {
    "Item": ["Pen", "Pen", "Book", "Book", "Pencil", "Pencil"],
    "Units_Sold": [10, 15, 20, 25, 30, 35]
}

df = pd.DataFrame(data)

# Create Pivot Table
pivot = pd.pivot_table(
    df,
    values="Units_Sold",
```

```
    index="Item",
    aggfunc="sum"
)

print(pivot)
```

OUTPUT

```
>>>
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/query processi
ng/exp 1 qp.py
       Units_Sold
Item
Book           45
Pen            25
Pencil         65
>>>
```

EXP 9

Aim

To write a Python program using Pandas to create a Pivot Table and find the total sale amount region-wise, manager-wise, and salesman-wise.

Algorithm

- ❖ Start the program.
- ❖ Import the Pandas library.
- ❖ Create a DataFrame with Region, Manager, SalesMan, and Sale_amt.
- ❖ Create a Pivot Table using `pd.pivot_table()`.
- ❖ Set Region, Manager, and SalesMan as the index.
- ❖ Use Sale_amt as the values and sum as the aggregation function.
- ❖ Display the Pivot Table.
- ❖ Stop the program.

CODE

```
import pandas as pd

# Sample sales data
data = {
    "Region": ["East", "Central", "Central", "West", "East"],
    "Manager": ["Martha", "Hermann", "Timothy", "Douglas", "Martha"],
    "SalesMan": ["Alexander", "Shelli", "David", "Michael", "Diana"],
    "Sale_amt": [113810, 25000, 6075, 38336, 14500]
}

df = pd.DataFrame(data)

# Create Pivot Table
pivot = pd.pivot_table(
    df,
    values="Sale_amt",
    index=["Region", "Manager", "SalesMan"],
    aggfunc="sum"
)

print(pivot)
```

OUTPUT

```
>>>
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/query processi
ng/exp 1 qp.py
                          Sale_amt
Region  Manager SalesMan
Central Hermann Shelli       25000
        Timothy David         6075
East    Martha  Alexander   113810
                Diana        14500
West    Douglas Michael      38336
```

EXP 10

Aim

To write a Python program using Pandas to create a DataFrame with 10 rows and 4 columns of random values and highlight negative numbers in red and positive numbers in black.

Algorithm

- Start the program.
- Import the Pandas and NumPy libraries.
- Create a DataFrame with random values.
- Define a function to color negative numbers red and positive numbers black.
- Apply the function using `style.map()`.
- Display the styled DataFrame.
- Stop the program.

CODE

```
import pandas as pd
import numpy as np

# Create DataFrame with random values
df = pd.DataFrame(
    np.random.randn(10, 4),
```

```
    columns=['A', 'B', 'C', 'D']
)
# Function to highlight values
def color(x):
    if x < 0:
        return 'color:red'
    else:
        return 'color:black'

# Display styled DataFrame
print(df)

# For Jupyter Notebook
df.style.map(color)
```

OUTPUT

```
>>>
= RESTART: C:/Users/udhay/AppData/Local/Programs/Python/Python311/query processi
ng/exp 1 qp.py
          A         B         C         D
0  0.469949 -0.157052 -0.156593 -0.638053
1  0.978953 -0.716054  0.614020  1.070386
2  0.965893  0.281230 -0.668425 -0.031761
3 -0.576278 -1.123108  1.506916  1.025364
4 -0.239889 -0.650365  0.375837 -0.946248
5 -0.218735 -1.379016  0.147394 -0.733952
6 -0.029388  0.516021  0.157386  1.395421
7 -0.060722  0.758564  0.169183 -0.247198
8  0.917315  0.401463 -0.200316  1.217789
9 -0.858231 -1.264342  1.443099  1.225444
>>>
```